
Rapport de soutenance finale

Projet Overdrive

Lucas Flacelière, Vasile Ionescu, Clément Bertrand, Mattieu
Bourgeois, Anatole Brunelle

Mardi 26 mai 2026

Table des matières:

1. Introduction

2. Participation de chaque membres

- 2.1 Lucas Flacelière
- 2.2 Vasile Ionescu
- 2.3 Clément Bertrand.....
- 2.4 Matthieu Bourgeois
- 2.5 Anatole Brunelle

3. Conclusion

1. Introduction

Ce rapport constitue le rendu final du projet Overdrive, réalisé dans le cadre de notre deuxième année de formation en ingénierie logicielle. Il présente de manière détaillée l'ensemble des contributions individuelles apportées par chacun des cinq membres de l'équipe au cours de ce semestre de développement. Ce document accompagne notre soutenance finale et vise à offrir une vision complète et transparente du travail accompli, des choix techniques effectués, des difficultés rencontrées et des solutions mises en œuvre.

Overdrive est un jeu de stratégie, de construction et de survie multijoueur d'inspiration steampunk, développé entièrement en Python à l'aide de la bibliothèque Pygame. Le concept central du jeu repose sur la construction et l'amélioration progressive d'une base industrielle dans un univers où des vagues de monstres viennent régulièrement menacer le village du joueur. Pour résister à ces assauts, le joueur doit gérer efficacement ses ressources — or, nourriture et vapeur —, déployer des bâtiments de production, de stockage et de défense, et coordonner ses villageois pour optimiser l'économie générale de sa cité. L'aspect multijoueur en réseau local permet à plusieurs joueurs de collaborer simultanément, ajoutant une dimension stratégique supplémentaire liée à la coordination entre les membres de l'équipe.

Ce projet a été mené par une équipe de cinq étudiants : Lucas Flacelère, Vasile Ionescu, Clément Bertrand, Matthieu Bourgeois et Anatole Brunelle. Dès le début du semestre, nous avons décidé de répartir le travail de manière claire et complémentaire, chaque membre prenant en charge un périmètre technique bien défini. Lucas s'est concentré sur l'intégralité du module réseau et la gestion du multijoueur. Vasile a pris en charge l'ambiance visuelle, les systèmes de ressources au sol, les PNJ et la partie sonore. Clément a travaillé sur la musique, le système d'amélioration des bâtiments, la tourelle et l'arbre de compétences. Matthieu a enrichi le contenu du jeu avec les centrales de boost, les bâtiments de stockage et les mécaniques de ressources évolutives. Enfin, Anatole a fondé l'architecture du jeu en refondant la grille de cases, en créant l'interface de gestion des villageois et en développant plusieurs outils de développement et d'ergonomie.

Sur le plan technique, le projet soulève des problématiques ambitieuses pour un développement réalisé en Python avec Pygame, une bibliothèque initialement conçue

pour des jeux simples et non pour des applications réseau ou des systèmes multi-agents complexes. L'équipe a donc dû dépasser les limites naturelles de cet environnement en implémentant de nombreux systèmes sur mesure : un protocole de communication TCP/UDP avec sérialisation JSON, un moteur de rendu à deux étapes avec surface intermédiaire, un système de pathfinding basé sur les routes, un arbre de compétences interactif et un pipeline d'animation par spritesheets. Chacune de ces fonctionnalités a demandé un travail de conception, de test et de débogage considérable, souvent en interaction avec les autres modules du jeu.

Ce rapport est structuré en cinq sections principales, correspondant chacune à la contribution d'un membre de l'équipe. Chaque section suit une organisation similaire : présentation du périmètre de contribution, description technique des fonctionnalités développées, analyse des problèmes rencontrés et des corrections apportées, et bilan personnel. Cette structure permet à chaque étudiant de valoriser son travail de manière autonome, tout en mettant en évidence la cohérence globale du projet. Nous espérons que ce document reflète fidèlement l'investissement de chacun et la qualité du résultat final obtenu.

2.1 Lucas Flacelière

2.1.1. Contexte et périmètre de la contribution

Contraintes du cahier des charges

L'obligation d'un mode multijoueur en réseau implique une architecture logicielle fondamentalement différente d'un programme purement local. Un programme capable de communiquer en réseau ne d'architecture pas du tout de la même manière : il faut gérer des threads concurrents, concevoir un protocole de communication, sérialiser des structures de données complexes et synchroniser l'état du jeu entre plusieurs machines indépendantes.

Ces contraintes ont donc été intégrées dès le début de la conception, conformément aux recommandations du cahier des charges qui souligne l'importance de penser le projet avec ces objectifs en tête dès le départ, surtout en ce qui concerne le réseau.

Périmètre de ma contribution

Mon périmètre couvre exclusivement :

-

2.1.2. Architecture générale du système réseau

Vue d'ensemble

Le module réseau est composé de trois fichiers principaux qui interagissent avec le reste du jeu :

- multiplayer/serveur.py — le serveur TCP, lancé sur la machine hôte

- multiplayer/client.py — le client TCP, exécuté sur chaque machine joueur
- screens/game_logic.py — la couche de synchronisation qui applique les mises à jour reçues à l'état local du jeu

Le serveur écoute sur le port TCP 5050 et utilise le port UDP 5051 pour diffuser sa présence sur le réseau local. Chaque client démarre en envoyant un broadcast UDP sur le sous-réseau ; dès qu'il reçoit une réponse du serveur, il récupère l'adresse IP et établit la connexion TCP.

Protocole de message

Chaque message transmis suit un format en deux parties : un en-tête de 64 octets contenant la taille du corps en clair (complété par des espaces), suivi d'un corps JSON de taille variable encodé en UTF-8. Ce système d'en-tête à taille fixe garantit que les messages peuvent être reconstitués même s'ils ont été fragmentés lors du transit TCP, un problème courant que les développeurs débutants négligent souvent.

Le corps JSON possède toujours un champ type qui détermine comment le destinataire interprète le champ payload. Les types supportés sont : str, int, float, bool, list, dict, tuple, bâtiment, liste_bâtiments et liste_joueurs. Ce typage explicite permet de démultiplexer facilement les messages reçus sans heuristiques fragiles.

2.1.3. Synchronisation des joueurs

2.1.3.1 Sérialisation de l'objet Player

La classe Player définit `to_dict()` et `from_dict()` pour sa sérialisation réseau. `to_dict()` exporte l'ensemble des attributs utiles à la synchronisation : position (pos), chemin de déplacement calculé par A* (path), état d'animation (anim_state, anim_frame, direction, is_moving), et attributs de jeu (hp, money, food, vapeur, speed). `from_dict()` reconstruit un objet Player complet avec des valeurs par défaut calquées sur `__init__` pour tolérer les paquets incomplets.

Un point délicat : pos et les éléments de path sont des tuples Python, type non sérialisable nativement en JSON. Ils doivent être convertis en listes avant l'encodage, puis reconvertis en tuples après décodage. Cette conversion est centralisée dans

send_liste_joueurs_client() côté client et dans handle_message_recieved() côté serveur.

2.1.3.2 Flux d'envoi

Les positions des joueurs sont envoyées depuis la boucle de jeu principale dans jeu.py, à chaque frame où le joueur local s'est déplacé :

```
if player.pos != prec:
    try:
        client_module.send_liste_joueurs_client(players, client_module.CLIENT)
    except:
        pass
```

Ce choix d'envoyer la liste entière (et non seulement le joueur local) simplifie la logique côté serveur : celui-ci reçoit un état complet et peut le redistribuer tel quel à tous les autres clients, sans avoir à fusionner des mises à jour partielles.

2.1.3.3 Flux de réception et affichage

Côté réception, game_logic.py met à jour la variable globale players dès qu'un message de type liste_joueurs est reçu. La fonction draw_players() parcourt cette liste et appelle draw_player() sur chaque objet, en appliquant la caméra et le zoom. Les animations des joueurs distants (marche, idle) sont pilotées par les champs anim_state et anim_frame reçus, sans recalcul local.

2.1.3.4 Gestion de la connexion et des indices

À chaque nouvelle connexion, le serveur crée un Player vide, l'ajoute à gl.players et diffuse la liste mise à jour à tous les clients. Il envoie également un entier indice à chaque client pour qu'il sache quelle entrée de la liste lui appartient. À la déconnexion, le serveur retire le Player de la liste, décrémente les indices des joueurs dont l'indice est supérieur à celui du partant, et redistribue la liste.

2.1.4. Synchronisation des bâtiments

2.1.4.1 Sérialisation de l'objet Batiment

La classe `Batiment` définit `to_dict()` qui exporte les champs `type`, `niveau`, `x`, `y`, `largeur` et `hauteur`. La position (`x`, `y`) est stockée en cases (coordonnées de grille), pas en pixels. C'est le `render` qui applique la multiplication par `TAILLE_CASE` pour obtenir les coordonnées écran. Ce choix garantit que les coordonnées restent correctes quelle que soit la taille de case utilisée par chaque client.

`from_dict()` reconstruit l'objet en appelant le constructeur `cls(type, x, y)`, ce qui déclenche le chargement des sprites et l'initialisation complète, puis écrase les attributs avec les valeurs du dictionnaire. Les champs `largeur` et `hauteur` sont relus depuis `DATA[type]` avec les clés `hitbox_l` et `hitbox_h` — et non depuis les clés du dictionnaire reçu, ce qui évite une incohérence si les deux noms divergent.

2.1.4.2 Flux d'envoi

Les bâtiments sont envoyés dans deux situations : lors de la pose d'un nouveau bâtiment, et lors de sa destruction ou amélioration. Dans les deux cas, c'est la liste complète bâtiments qui est envoyée, pas un delta.

```
batiments.append(nouveau)
sound.son_placement.play()
gl.synchroniser_npcs(batiments, npcs, players[indice], TAILLE_CASE)
if client_module.CLIENT is not None and online:
    client_module.send_liste_batiments_client(batiments, client_module.CLIENT)
```

La même séquence est exécutée après une destruction (`batiments.remove()`) et après une amélioration de niveau. Cela garantit que tous les clients disposent en permanence d'une copie cohérente de la carte de construction du joueur hôte.

2.1.4.3 Flux de réception

Côté client, `game_logic.py` reçoit la liste et l'affecte à la variable globale `batiments`. Une ligne supplémentaire force `en_construction = False` sur chaque bâtiment reçu, pour s'assurer que les clients distants n'affichent pas l'animation de construction en cours (qui n'a de sens que pour le joueur qui vient de poser le bâtiment).


```
elif msg_type == "liste_batiments":  
    batiments = message  
    for batiment in batiments:  
        batiment.en_construction = False
```

2.1.4.4 Envoi initial à la connexion

Lorsqu'un nouveau client se connecte alors qu'une partie est déjà en cours, le serveur lui envoie immédiatement la liste des bâtiments existants stockée dans `gl.batiments`. Ce mécanisme est déclenché dans `handle_client()` dès que `number_connected > 1`, ce qui permet au nouveau joueur d'afficher la carte du joueur hôte sans délai.

2.1.5. Synchronisation des monstres et des raids

2.1.5.1 Architecture : le RaidManager et la notion de leader

Les monstres sont gérés par la classe `RaidManager` dans `core/pve.py`. Cette classe orchestre les vagues (waves), gère le spawn des monstres autour des joueurs et met à jour leur IA à chaque frame. En multijoueur, une seule instance fait autorité sur les monstres : le joueur hôte, identifié par le flag `leader = True`. Les clients reçoivent la liste des monstres et le `RaidManager` sérialisé, mais ne calculent pas eux-mêmes le spawn ni la logique de vague.

Ce choix évite la désynchronisation qui surviendrait si chaque client générerait ses propres monstres indépendamment. Seul le leader spawne, met à jour et envoie ; les autres affichent uniquement.

2.1.5.2 Sérialisation de Monster

`Monster.to_dict()` exporte la position (`x`, `y`), les points de vie (`hp`), l'état de vie (`alive`), les timers d'animation (`_anim_frame`, `_anim_timer`, `_anim_phase`, `_facing_left`) et les timers d'attaque (`_attack_timer`, `_is_attacking`, `_attack_anim_timer`, `_attack_damage_applied`). `Monster.from_dict()` appelle le constructeur avec `x` et `y` pour déclencher le chargement des sprites, puis écrase tous les attributs.

Les champs `damage_numbers` (chiffres de dégâts flottants) ne sont pas sérialisés : ce sont des effets visuels éphémères, sans intérêt pour la synchronisation réseau.

2.1.5.3 Flux d'envoi des monstres

Les monstres sont envoyés depuis deux endroits dans `pve.py`. D'abord, dans la méthode `update()`, à chaque frame où la liste de monstres change (mort d'un monstre) :

```
self.monsters = [m for m in self.monsters if m.alive]

if self.monsters != old_monsters:
    try:
        client_module.send_liste_monstres_client(self.monsters, client_module.CLIENT)
    except Exception as e:
        pass
```

Ensuite, dans `_spawn_wave()`, après chaque apparition d'une vague, le `RaidManager` complet est envoyé via `send_raid_client()`. Cela permet aux clients de synchroniser non seulement la liste de monstres mais aussi l'état interne du raid (numéro de vague, timers, compteur de raids).

2.1.5.4 Flux de réception

Le serveur reçoit les messages `liste_monstres` dans `handle_message_recieved()` et les passe à `handle_monsters()`. Cette fonction implémente une logique de résolution de conflits : le premier client à se connecter est désigné `monster_director`. Seul le `monster_director` peut augmenter la taille de la liste (`spawn`) ; n'importe quel client peut la réduire (mort). La liste fusionnée est ensuite redistribuée à tous les autres clients.

```

monsters = []
monster_director = None
def handle_monsters(liste_monsters, client): 1 usage 2 lucas.F
    global monsters, monster_director
    if monster_director is None:
        monster_director = client
    if len(liste_monsters) > len(monsters):
        if monster_director == client:
            monsters = liste_monsters
    elif len(liste_monsters) < len(monsters):
        monsters = liste_monsters
    for Client in clients.keys():
        if clients[Client] != client:
            try :
                send_liste_monstres(monsters, clients[Client])
            except:
                pass
    return monsters

```

Côté client, game_logic.py reçoit les messages liste_monstres et raid et met à jour respectivement gl.monsters et jeu.raid_manager. Lorsqu'un RaidManager est reçu, son flag leader est forcé à False, pour que le client ne déclenche pas lui-même de nouvelles vagues.

2.1.6. Problèmes rencontrés et corrections apportées

2.1.6.1 Condition inversée dans disconnect()

Lors d'une déconnexion, le serveur doit décrémenter l'indice de tous les joueurs dont l'indice est strictement supérieur à celui du joueur partant. La version initiale utilisait la condition $< i$ au lieu de $> i$, ce qui décrémentait les joueurs précédant le partant. La liste se désynchronisait immédiatement, provoquant un IndexError à la déconnexion suivante.

2.1.6.2 Mauvais argument dans send_liste_joueurs_client

Dans la méthode `update()` de `Player`, la fonction réseau était appelée avec `client_module` (le module entier) au lieu de `client_module.CLIENT` (le socket actif). L'erreur était avalée par un bloc `except` générique, ce qui la rendait totalement silencieuse. Les positions n'étaient jamais transmises au serveur, et les autres joueurs restaient figés.

2.1.6.3 Sérialisation partielle pos/path

La conversion des tuples Python en listes JSON-compatibles n'était appliquée qu'au premier élément de la liste de joueurs (`payload[0]`). Avec trois joueurs ou plus, la sérialisation JSON échouait pour les joueurs aux indices 1 et suivants. Ce bug a été corrigé en introduisant une fonction `serialize_joueurs()` qui itère sur l'ensemble de la liste, utilisée dans les trois sites d'envoi concernés.

2.1.6.4 Tiles de bâtiments cinq fois trop grandes chez les clients

`from_dict()` de `Batiment` reconstruisait largeur et hauteur en cherchant les clés "largeur" et "hauteur" dans `DATA`, qui n'existent pas. Python retournait `DEFAULT_FOOTPRINT = 5` pour tous les bâtiments. Comme le `render` multiplie ces valeurs par `TAILLE_CASE`, les bâtiments étaient systématiquement cinq fois trop grands. La correction utilise les mêmes clés que `__init__` : `hitbox_l` et `hitbox_h`.

2.1.6.5 Comparaison d'objets sans __eq__ dans on_message_recu

La variable `messageprec` était comparée au message reçu avec `!=`. Les listes d'objets `Player` sans `__eq__` rendent cette comparaison non fiable. La correction remplace la comparaison de valeur par une comparaison d'identité Python (`current is not result_prec`), garantissant que tout nouveau tuple résultat est traité, quelle que soit la nature de son contenu.

2.1.7. Bilan et perspectives

2.1.7.1 Ce qui fonctionne

- Connexion automatique sur réseau local sans configuration manuelle (broadcast UDP)
- Synchronisation en temps réel des positions et animations de 2 à 4 joueurs
- Propagation instantanée des bâtiments posés ou détruits à tous les clients connectés
- Synchronisation des monstres et de l'état du raid avec gestion de l'autorité (leader/monster_director)
- Déconnexion propre sans crash du serveur ni des clients restants

2.1.7.2 Difficultés rencontrées

Le débogage de systèmes concurrents est particulièrement éprouvant : les bugs sont intermittents, non reproductibles facilement, et souvent silencieux (exceptions avalées, comparaisons incorrectes sans erreur). Plusieurs anomalies n'ont été détectées qu'en observant des comportements visuels inattendus, par exemple des joueurs figés ou des bâtiments de taille incorrecte.

La coordination avec les autres membres de l'équipe a également demandé une discipline de conception : le module réseau devait rester le plus transparent possible pour ne pas bloquer l'avancement des fonctionnalités développées en parallèle.

2.1.7.3 Pistes d'amélioration

- Interpolation côté client pour lisser les mouvements lors de pics de latence
- Envoi de deltas plutôt que de listes complètes pour réduire la bande passante
- Validation des payloads reçus pour protéger le serveur contre des messages malformés
- Protocole UDP pour les mises à jour de position, mieux adapté aux données temps réel

2.1.7.4 Compétences acquises

Ce projet m'a permis de travailler concrètement avec les sockets TCP et UDP Python, les threads et leur synchronisation, la conception de protocoles applicatifs binaires, la sérialisation de graphes d'objets complexes et le débogage de systèmes distribués. Travailler seul sur ce module tout en m'intégrant à une équipe m'a également appris à concevoir des interfaces claires entre les couches d'un logiciel, compétence centrale dans tout projet collaboratif de développement.

2.2 Vasile Ionescu

Développement des systèmes d'ambiance et d'exploration

Mise en place du système de nuages

Au cours de cette phase du projet, j'ai commencé à travailler davantage sur l'ambiance générale du jeu. Même si les mécaniques principales fonctionnaient déjà correctement, la map paraissait encore vide et statique. J'ai donc voulu ajouter des éléments capables de rendre l'environnement plus vivant afin que le joueur ait réellement l'impression d'évoluer dans un monde dynamique.

La première fonctionnalité développée a été un système de nuages dynamiques. Mon objectif était de créer un mouvement permanent dans le décor sans gêner la visibilité du joueur. J'ai créé un gestionnaire capable de générer plusieurs nuages à des positions aléatoires sur toute la map. Chaque nuage possède une taille, une vitesse et une position légèrement différentes afin d'éviter un rendu répétitif.

L'implémentation a été plus compliquée que prévu à cause du système de caméra. Comme le joueur peut se déplacer librement dans le monde, il fallait constamment convertir les coordonnées du monde vers celles affichées à l'écran. Au début, les nuages semblaient glisser lorsque le joueur bougeait, ce qui donnait un rendu peu naturel. J'ai donc dû reprendre plusieurs fois les calculs liés aux offsets de caméra avant d'obtenir un déplacement fluide.

J'ai également ajouté un système de réapparition automatique afin que les nuages soient replacés lorsqu'ils quittent la zone visible de la map. Cela permet de garder un ciel vivant en permanence sans devoir générer de nouveaux objets continuellement.

Création du système de ressources au sol

Après avoir travaillé sur l'ambiance visuelle, j'ai développé un système de ressources réparties sur toute la map afin d'encourager l'exploration. Mon objectif était d'éviter que le joueur reste uniquement autour de sa base pendant toute la partie.

J'ai créé trois types principaux de ressources :

- le bois ;
- les herbes ;
- les coffres.

Le bois et les herbes permettent de récupérer une quantité aléatoire de nourriture tandis que les coffres donnent de l'argent avec une chance d'obtenir des objets rares comme des potions.

Pour implémenter ce système, j'ai créé une structure de données contenant toutes les ressources présentes dans le monde avec leur type et leurs coordonnées. Lors du lancement d'une partie, plusieurs ressources sont générées automatiquement sur une très grande zone autour du joueur afin de rendre chaque partie légèrement différente.

Cette partie du développement m'a demandé énormément de tests. Au début, les ressources apparaissaient soit beaucoup trop proches, soit beaucoup trop éloignées. Dans certains cas, plusieurs coffres apparaissaient presque au même endroit, ce qui

rendait l'exploration inutile. À l'inverse, certaines zones étaient totalement vides. J'ai donc dû ajuster les paramètres de génération afin d'obtenir une répartition plus naturelle.

J'ai aussi ajouté plusieurs vérifications pour empêcher les ressources d'apparaître dans des zones problématiques comme la zone de départ du joueur ou directement à l'intérieur des bâtiments. Sans ces vérifications, certaines ressources devenaient impossibles à récupérer.

Enfin, j'ai développé un système de collecte interactif. Lorsque le joueur clique sur une ressource, le jeu vérifie s'il est suffisamment proche pour la récupérer. Si c'est le cas, la ressource disparaît et le joueur reçoit automatiquement sa récompense avec un affichage visuel des gains obtenus.

J'ai également réfléchi à l'intérêt du système sur le long terme. Je ne voulais pas que les ressources disparaissent définitivement après quelques minutes de jeu. J'ai donc ajouté un système de réapparition progressive permettant de maintenir la map active et intéressante pendant toute la partie.

Développement des mécaniques de progression

Création du système de potions

Après avoir terminé le système de ressources, je me suis rendu compte qu'il manquait encore une vraie raison d'explorer la map sur le long terme. J'ai donc décidé de créer un système de potions afin d'ajouter davantage de progression et de récompenses rares.

Les coffres peuvent maintenant contenir plusieurs types de potions offrant des bonus temporaires au joueur. Certaines permettent de doubler les gains d'argent, d'autres augmentent la nourriture récupérée ou améliorent la production de vapeur. J'ai également ajouté une potion de soin permettant de récupérer progressivement des points de vie.

L'implémentation de ce système a nécessité la création d'un système de timers permettant d'activer automatiquement les bonus pendant une durée précise avant leur disparition. Cette partie a demandé beaucoup plus de travail que prévu car il fallait constamment vérifier les effets actifs sans provoquer de bugs.

J'ai également dû corriger plusieurs problèmes liés au cumul des effets. Au départ, certaines potions pouvaient se superposer et rendaient le jeu totalement déséquilibré. J'ai finalement choisi de prolonger simplement la durée d'un effet lorsqu'une potion identique est réutilisée.

J'ai aussi dû équilibrer les statistiques des potions. Pendant certains tests, les bonus étaient tellement puissants que le joueur pouvait accumuler énormément de ressources en quelques minutes. J'ai donc réduit plusieurs valeurs afin de conserver une progression cohérente.

Je voulais également que les potions restent simples à comprendre pour le joueur. J'ai donc essayé de créer des effets faciles à identifier visuellement afin que le joueur comprenne immédiatement l'intérêt d'une potion lorsqu'il la récupère.

Développement de l'inventaire

Une fois le système de potions terminé, j'ai créé un inventaire permettant au joueur de stocker et utiliser les objets récupérés.

L'inventaire fonctionne sous forme de grille contenant plusieurs slots affichant les potions, leur image ainsi que leur quantité disponible. Lorsqu'un joueur clique sur une potion, le jeu vérifie automatiquement s'il possède encore un exemplaire avant d'activer son effet.

Cette partie m'a demandé de nombreux ajustements visuels. Les textes, les images et les slots étaient parfois mal alignés ou mal dimensionnés. J'ai donc dû retravailler plusieurs éléments de l'interface afin d'obtenir un rendu propre et lisible.

J'ai également dû gérer les interactions entre l'inventaire et les effets actifs. Il fallait que les bonus soient correctement synchronisés avec l'interface afin que le joueur puisse voir immédiatement les changements.

Ce système de potions et d'inventaire a fortement amélioré l'intérêt de l'exploration puisque les coffres sont devenus des récompenses réellement utiles pour la progression du joueur.

Développement du système de déplacement des villageois

Création du système de routes avec les tiles

Après avoir ajouté les systèmes de ressources et de progression, j'ai commencé à travailler sur l'intelligence artificielle des villageois. Mon objectif était que les PNJ puissent se déplacer de manière logique dans la ville.

J'ai donc créé un système de routes basé sur des tiles. Le principe est que les villageois ne peuvent rejoindre leur lieu de travail que si un chemin de routes relie leur maison au bâtiment correspondant.

Pour cela, j'ai ajouté un nouveau type de bâtiment représentant les routes du jeu. Chaque tile correspond à une case de la map et peut se connecter aux autres tiles voisines.

Au début, les PNJ se déplaçaient simplement en ligne droite vers leur destination, ce qui provoquait de nombreux problèmes. Ils traversaient les bâtiments, ignoraient totalement les routes et donnaient un comportement peu réaliste.

J'ai donc entièrement revu le système de déplacement. Désormais, lorsqu'un villageois reçoit un travail, il recherche d'abord une tile située autour de sa maison puis vérifie progressivement si un chemin complet existe jusqu'au bâtiment de production.

Si aucun chemin n'est trouvé, le PNJ retourne temporairement en errance autour de sa maison avant de recommencer automatiquement la recherche quelques secondes plus tard.

Cette partie a demandé énormément de corrections car certains villageois considéraient qu'aucun chemin n'existait alors qu'une route correcte était pourtant présente. J'ai donc ajouté davantage de vérifications et finalement refait presque entièrement le système de pathfinding.

Gestion des comportements des PNJ

J'ai également créé plusieurs états pour les villageois :

- errance ;
- recherche de chemin ;
- déplacement vers le travail ;
- travail ;
- retour à la maison.

Chaque état possède sa propre logique et ses propres vérifications. Cette organisation m'a permis d'éviter de nombreux blocages et de rendre les comportements beaucoup plus crédibles.

Le plus compliqué a été la gestion des transitions entre les états. Dans certaines situations, les PNJ passaient constamment d'un état à l'autre sans jamais atteindre leur destination. J'ai donc dû ajouter plusieurs sécurités afin d'éviter ces boucles de comportement.

J'ai aussi ajouté un système de vérification continue : si un nouveau chemin est construit par le joueur, le PNJ peut désormais le détecter automatiquement et reprendre son trajet normalement.

Cette fonctionnalité a fortement amélioré le gameplay car le joueur doit maintenant réellement réfléchir à l'organisation de sa ville et au placement des routes pour optimiser le fonctionnement de ses bâtiments.

Travail sur l'immersion visuelle et sonore

Ajout du vent et des feuilles animées

Une fois les mécaniques principales terminées, j'ai voulu améliorer davantage l'ambiance du jeu. J'ai donc développé un système de vent accompagné de feuilles animées traversant la map.

Chaque feuille possède sa propre vitesse, sa propre rotation ainsi qu'un léger mouvement ondulé afin de simuler l'effet du vent. J'ai également ajouté des particules de vent très légères afin de rendre l'environnement plus vivant.

Cette fonctionnalité a demandé plusieurs essais avant d'obtenir un résultat naturel. Les feuilles étaient parfois trop rapides ou au contraire presque immobiles. J'ai donc dû ajuster les vitesses et les trajectoires afin de trouver un bon équilibre.

Même si cette fonctionnalité paraît secondaire, elle améliore énormément l'ambiance générale du jeu et donne l'impression que le monde continue de vivre même lorsque le joueur ne fait rien.

Mise en place des visuels des bâtiments

Pour améliorer l'aspect graphique du projet, nous avons acheté une grande plaquette contenant de nombreux assets pixel art représentant différents bâtiments.

Les assets n'étaient cependant pas directement exploitables dans le jeu. J'ai donc dû découper manuellement chaque bâtiment puis les redimensionner afin qu'ils correspondent correctement au système de cases de la map.

Cette étape a demandé beaucoup de temps car certains sprites étaient mal alignés ou contenaient beaucoup d'espace vide autour d'eux. J'ai aussi dû gérer différentes tailles de bâtiments afin qu'ils restent cohérents visuellement.

J'ai également corrigé plusieurs problèmes de qualité visuelle liés au redimensionnement des images. Certaines méthodes rendaient les sprites flous, ce qui ne convenait pas au style pixel art du jeu.

J'ai aussi passé du temps à harmoniser les couleurs et les tailles afin que tous les bâtiments semblent appartenir au même univers graphique.

Implémentation des spritesheets des PNJ

J'ai ensuite travaillé sur les animations des villageois en implémentant de véritables spritesheets.

Chaque spritesheet contient plusieurs frames représentant les différentes étapes de déplacement des PNJ. J'ai dû découper chaque frame puis créer un système capable d'afficher automatiquement la bonne animation selon le mouvement du personnage.

J'ai aussi utilisé un système de retournement horizontal afin de générer automatiquement certaines directions à partir d'une seule animation, ce qui m'a permis de gagner du temps.

Cette partie m'a demandé plusieurs corrections car certaines animations n'étaient pas correctement alignées. Certains personnages semblaient légèrement glisser ou sauter pendant leurs déplacements. J'ai donc dû ajuster précisément les positions des sprites.

Travail sur l'ambiance sonore et l'interface utilisateur

Intégration des effets sonores

Après avoir amélioré les éléments visuels du jeu, j'ai commencé à travailler sur toute la partie sonore afin de rendre l'expérience plus immersive.

J'ai ajouté plusieurs effets sonores correspondant aux principales actions du jeu :

- les pas du joueur ;
- la collecte des ressources ;
- les tirs des tourelles ;
- la musique d'ambiance.

L'intégration des sons de pas a été particulièrement compliquée. Au début, plusieurs sons se jouaient en même temps lorsque le joueur changeait rapidement de direction. J'ai donc dû modifier entièrement la logique de lecture afin que le son ne se relance pas constamment.

J'ai également recherché de nombreux effets sonores sur des banques libres de droit. J'ai testé plusieurs dizaines de sons avant de trouver des effets correspondant réellement à l'ambiance du jeu. Certains ont ensuite été raccourcis ou modifiés afin qu'ils correspondent mieux au rythme du gameplay.

J'ai aussi ajouté un son de tir pour les tourelles accompagné d'un effet visuel d'explosion placé au bout du canon. Cette fonctionnalité améliore fortement le ressenti pendant les combats.

La partie sonore m'a demandé beaucoup plus de travail que prévu car il fallait trouver des sons cohérents entre eux afin de garder une identité sonore homogène.

Ajout des informations dans l'interface de construction

J'ai aussi amélioré l'interface de construction des bâtiments en ajoutant directement leurs noms ainsi que leurs prix sous les icônes.

Cette modification a demandé plusieurs ajustements liés aux tailles de texte, aux espacements et aux alignements afin que l'interface reste claire et lisible.

J'ai également ajouté plusieurs effets visuels permettant de mieux identifier le bâtiment actuellement sélectionné.

Corrections, optimisation et résolution des bugs

Refonte du pathfinding des villageois

La plus grande difficulté technique de cette phase du projet a été le système de déplacement des villageois. Même après la création des routes, de nombreux problèmes continuaient d'apparaître pendant les tests.

Les PNJ restaient parfois bloqués sans raison ou considéraient qu'aucun chemin n'existait alors qu'une route correcte était pourtant présente. J'ai donc entièrement revu le système de pathfinding afin que les villageois analysent correctement toutes les tiles voisines et retestent régulièrement les chemins disponibles.

Cette nouvelle logique a permis de rendre les comportements beaucoup plus stables et cohérents.

Correction du système des tourelles

Une autre difficulté importante concernait les tourelles. Pendant les tests, elles tuaient parfois les ennemis instantanément alors que les dégâts définis dans le code étaient pourtant faibles.

Le problème venait du fait que les dégâts étaient appliqués plusieurs fois à cause d'une erreur dans la logique d'attaque. J'ai donc repris entièrement le système de tir afin de corriger ce problème et d'ajouter un cooldown plus propre.

J'en ai également profité pour améliorer les effets visuels des tirs avec un effet d'explosion placé au bout du canon et orienté automatiquement selon la direction de la cible.

Corrections visuelles et ajustements divers

Enfin, une grande partie du travail a consisté à corriger de nombreux petits problèmes visuels présents dans le jeu.

J'ai notamment corrigé plusieurs problèmes d'alignement dans l'interface utilisateur, ajusté la taille de certaines ressources et repositionné différents sprites afin qu'ils correspondent correctement aux cases de la map.

J'ai aussi corrigé plusieurs problèmes liés au redimensionnement des images afin de conserver un rendu pixel art propre et net.

Cette phase de correction a été essentielle car elle m'a permis de rendre le jeu beaucoup plus stable, agréable à utiliser et cohérent visuellement.

Phase de tests et d'optimisation générale

Une partie importante du travail a également consisté à effectuer des tests réguliers afin de vérifier la stabilité générale du projet. Même lorsqu'une fonctionnalité semblait terminée, de nouveaux problèmes apparaissaient souvent lorsque plusieurs systèmes interagissaient ensemble.

Par exemple, certains bugs n'apparaissaient que lorsque plusieurs bâtiments étaient placés rapidement ou lorsque plusieurs villageois essayaient d'utiliser les mêmes routes en même temps. J'ai donc dû créer de nombreuses situations de test afin d'observer précisément le comportement du jeu dans différents cas.

J'ai également travaillé sur l'optimisation générale des performances. Comme la map contient beaucoup d'éléments affichés en permanence, notamment les ressources, les particules de vent, les feuilles, les bâtiments et les PNJ, certaines parties du jeu commençaient à ralentir lorsque trop d'objets étaient présents à l'écran. J'ai donc optimisé plusieurs systèmes afin d'éviter des calculs inutiles et de limiter l'affichage uniquement aux éléments visibles par la caméra.

Une autre difficulté concernait la synchronisation entre les différents systèmes du jeu. Par exemple, certaines animations continuaient à se jouer alors qu'un bâtiment était détruit ou qu'un PNJ avait changé d'état. J'ai donc ajouté plusieurs vérifications supplémentaires afin que chaque élément du jeu soit correctement mis à jour en temps réel.

J'ai aussi passé beaucoup de temps à améliorer le confort général du joueur. Même de petits détails comme la taille des textes, le positionnement des icônes ou la vitesse de certaines animations ont demandé plusieurs ajustements. Ces modifications paraissent parfois mineures individuellement, mais elles améliorent énormément la qualité globale du jeu et rendent l'expérience beaucoup plus agréable.

Enfin, cette phase de développement m'a permis de mieux comprendre l'importance des tests dans la création d'un jeu vidéo. Une fonctionnalité peut sembler correcte lorsqu'elle est développée seule, mais provoquer de nombreux problèmes lorsqu'elle interagit avec le reste du projet. J'ai donc appris à tester régulièrement chaque modification avant d'ajouter de nouvelles fonctionnalités.

2.3 Clément Bertrand

Mes tâches principales étaient:

- L'univers/ambiance sonore du jeu
- Les mécaniques d'amélioration des bâtiments
- Le bâtiment défensif et son fonctionnement
- la partie fonctionnelle de l'arbre de compétence
- le visuel du sol grandissant en fonction des déplacements

L'univers et l'ambiance musicale

La musique et l'environnement sonore sont des piliers essentiels pour garantir l'immersion totale du joueur dans notre univers. Notre objectif principal est de créer une atmosphère de détente absolue, offrant un véritable moment de relaxation. Aujourd'hui, nous pensons avoir atteint cet objectif. Comme pour la première phase du projet, j'ai pris en charge la composition musicale et la création des effets sonores (SFX), en faisant en sorte que chaque bruitage participe à cette atmosphère de sérénité.

Les multiples thèmes musicaux qui accompagnent désormais les différents écrans et phases de jeu ont tous été pensés et finalisés pour être les plus agréables et enveloppants possible, évitant toute fatigue auditive.

Ces musiques ont en partie été trouvées et sélectionnées sur Internet et composées par moi même. On trouve en ligne de nombreuses banques gratuite de musique d'ambiance, de thème musicaux, ou de bruitage. Le travail a donc été de sélectionner les meilleurs afin d'accomplir nos objectifs. Ce sont les suivantes que nous avons principalement utilisées, elles ont toutes une utilisation gratuite et on y a trouvé ce que l'on cherchait pour les musiques manquantes de notre jeu. :

- Pixabay Music
- Free Music Archive
- La Bibliothèque Audio YouTube

Parlons maintenant des critères de sélection/création de ces musiques.

Pour un jeu de construction axé sur la sérénité et la relaxation, le choix de la bande-son ne pouvait pas être laissé au hasard. Comme mentionné précédemment, la musique ne doit pas simplement combler le silence ; elle doit servir à apaiser le joueur. Pour y parvenir, nous avons défini des critères de sélection stricts basés sur nos expériences respectives en tant que joueur, nous avons pris exemple sur des jeux à succès dans ce style auxquels nous avons joué.

1. La Texture Sonore

Le premier critère technique a été l'élimination de toutes les agressions acoustiques. Nous avons banni les rythmiques trop percutantes, les basses lourdes et les instruments aux sonorités trop criardes. À l'inverse, nous avons privilégié des textures sonores douces et enveloppantes : des nappes de synthétiseurs douces, des notes de piano, ou encore des instruments acoustiques à cordes frottées ou pincées. Ces timbres ont la particularité de s'effacer naturellement à l'arrière-plan, offrant un soutien émotionnel chaleureux sans jamais accaparer l'attention du joueur.

2. Le Tempo/Rythme

Le rythme d'un jeu de construction calme doit inviter à prendre son temps. Notre second critère s'est donc porté sur le tempo. Nous avons sélectionné des morceaux au tempo lent ou modéré (généralement situés entre 60 et 80 battements par minute), ce qui correspond au rythme cardiaque d'un corps au repos. L'absence de structure rythmique rigide ou l'utilisation de rythmes très subtils permet au joueur de planifier et de bâtir ses structures à son propre rythme, sans ressentir la moindre pression temporelle ou sensation d'urgence.

3. Des musiques neutre pour éviter la lassitude ou l'excitation

Dans un jeu de gestion ou de construction, le joueur passe de longues heures sur les mêmes écrans. Le piège principal est la lassitude, voire l'irritation auditive. Pour éviter cela, nous avons recherché une forme de neutralité harmonique. Les compositions sélectionnées évitent les progressions d'accords trop mélancoliques (qui pourraient rendre le jeu triste) ou trop triomphantes (qui briseraient le calme). En privilégiant des structures musicales cycliques, minimalistes et progressives, la musique devient un flux continu et rassurant, capable de tourner en boucle de manière presque imperceptible pour l'oreille humaine.

Nous avons également intégré un système de rotation dynamique des sons. Concrètement :

- Nous avons plusieurs musiques disponibles et le jeu les joue de manière aléatoire.

Cette approche permet de briser la monotonie, d'éviter la lassitude auditive et d'offrir une expérience de jeu fluide, naturelle et profondément relaxante du début à la fin.

L'Importance Cruciale des Effets Sonores (SFX)

Au-delà de la bande-son, nous avons accordé une attention toute particulière aux effets sonores. C'est un aspect du développement de jeu vidéo qui est souvent sous-estimé ou qui passe inaperçu aux yeux des joueurs, alors qu'il est indispensable pour donner vie aux interactions. Pour transmettre l'atmosphère apaisante que nous visions, il était impensable de ne pas négliger ces détails. Un bon effet sonore ne doit

pas seulement être entendu, il doit être ressenti et s'intégrer harmonieusement dans l'expérience globale sans jamais agresser l'oreille. On pourrait même dire qu'il ne faut pas l'entendre, car cela signifierait qu'il n'est pas totalement à sa place.

Il nous faut des sons qui sont légers, qui n'attaquent pas les oreilles et surtout qui sont écoutables de nombreuses fois d'affilée sans qu'ils deviennent irritants. Ce dernier point est capital car, pour les sons de placement de bâtiment, d'amélioration ou de bruits de pas, le joueur va les entendre de nombreuses fois et souvent très rapidement d'affilée. Il faut donc que ces sons ne deviennent pas durs à écouter et qu'ils ne provoquent pas d'overdose sensorielle.

La fonction d'amélioration des bâtiments

Sur le plan fonctionnel, chaque bâtiment posé sur la map passe par plusieurs états :

- L'état de chantier : Dès qu'on pose le bâtiment, il passe en mode construction. Pendant ce temps-là, il bloque une case sur la grille mais il ne produit rien.
- L'état actif (Niveau 1) : Une fois le chrono terminé, le bâtiment commence enfin à fonctionner avec ses stats de base.
- L'évolution de niveau : Quand le joueur clique sur le bâtiment, une interface s'ouvre pour lui permettre de dépenser de l'or afin de le faire passer au niveau supérieur (Niveau 2 puis Niveau 3).

Pour l'équilibrage, le but était d'inciter le joueur à améliorer ses bâtiments plutôt qu'à en poser partout sur la carte pour saturer l'espace. Améliorer une structure existante coûte un peu plus cher, mais c'est beaucoup plus rentable en termes de place et de rendement de ressources. De plus, on a lié ce système à l'arbre des compétences : si le joueur n'a pas débloqué la technologie de la maison au niveau 2 dans son arbre, le jeu l'empêche de l'améliorer, ce qui force une vraie cohérence dans sa stratégie de partie.

Pour coder ça proprement sans faire ramer le jeu et en évitant d'écrire des tonnes de lignes répétitives, on a utilisé la programmation orientée objet de Python, combinée à des dictionnaires de configuration.

Le dictionnaire central DATA

Au lieu de faire des blocs de conditions if/else interminables pour savoir combien coûte l'amélioration de tel ou tel bâtiment à tel niveau, on a tout centralisé dans un dictionnaire de classe static appelé DATA dans notre classe Bâtiment. Ce dictionnaire regroupe toutes les données fixes : la taille de la hitbox (hitbox_l et hitbox_h), l'échelle d'affichage, le coût en or, le temps de construction en millisecondes et ce que rapporte le niveau (les dégâts pour une tourelle, la population pour une maison, etc.).

L'énorme avantage de cette méthode en Python, c'est que ça nous a facilité la vie pour l'équilibrage pendant nos sessions de test. Si on trouvait qu'une mine de niveau 2 produisait trop d'or ou qu'une ferme mettait trop de temps à se construire, on avait juste à changer un chiffre dans le dictionnaire au début du fichier, sans avoir besoin de retoucher à la logique de nos fonctions de calcul.

Cela nous a été particulièrement utile lorsqu'il a fallu régler et implanter les images par rapport à la taille réelle de leur hitbox. Les images ne pouvant pas être distordues, il a fallu trouver des équilibres parfaits pour que les images se "calent" parfaitement sur la grille que nous avons mise en place. Nous avons donc jonglé entre augmenter la taille de l'image (avec un coefficient multiplicateur) et la taille de sa "footprint" sa taille réelle sur le monde qui sera prise en compte pour les collisions.

Variables d'état et sérialisation

Chaque fois qu'un bâtiment est créé, l'objet possède ses propres variables locales : son type, ses coordonnées (x, y) sur la grille, et son niveau actuel. On utilise aussi des variables comme self.debut_construction pour stocker le moment exact où l'action a été lancée.

Pour que le code reste propre, tout est encapsulé dans des méthodes simples comme upgrade(), get_stats() ou get_upgrade_cost(). Ces fonctions vérifient l'état de l'objet et interrogent un autre module (get_max_level) pour savoir si le niveau demandé respecte le niveau maximum débloqué dans l'arbre. Enfin, comme on a une

partie multijoueur dans le projet, on a créé une méthode `to_dict()` qui transforme toutes les données d'un bâtiment en un dictionnaire Python standard. C'est pratique pour envoyer l'état des constructions au serveur et synchroniser tous les clients connectés.

Le calcul du temps de construction

Pour ne pas bloquer le jeu pendant qu'un bâtiment se construit ou s'améliore, on utilise l'horloge globale de Pygame avec `pygame.time.get_ticks()`, qui donne le temps écoulé en millisecondes depuis le lancement du programme. Quand le joueur lance une amélioration, on capture le temps de départ. Ensuite, dans la boucle principale du jeu, la méthode `construction_finie()` fait le calcul à chaque frame :

Si ce temps est supérieur ou égal à la durée requise (par exemple 5000 ms), le bâtiment passe enfin au niveau suivant. Pendant ce temps-là, on fait une simple division pour obtenir un pourcentage entre 0% et 100%. Ce chiffre nous sert à dessiner une barre de progression et à faire tourner la sprite d'engrenage au-dessus du bâtiment. Tout se fait en arrière-plan sans bloquer les déplacements du joueur ni les attaques des monstres.

Création procédurale de l'interface graphique (GUI)

Notre classe `MenuAmelioration` gère l'affichage du panneau d'upgrade dès que le joueur clique sur un bâtiment. Pour rester dans le thème Steampunk de notre jeu et éviter d'avoir à charger des images pour l'interface. N'ayant pas assez de compétences en graphisme et ne voulant pas tout chercher sur Internet, nous avons décidé de tout dessiner directement en code avec les outils géométriques de Pygame.

On utilise `pygame.draw.rect` avec l'option `border_radius` pour faire des fenêtres aux bords arrondis. On applique ensuite des bordures de couleurs différentes (un mélange de couleurs Cuivre et Laiton) pour donner du relief. Pour afficher le niveau actuel, on dessine trois petits cercles (`pygame.draw.circle`) alignés, qui se remplissent en jaune-or selon le niveau du bâtiment.

Pour les textes, on utilise `pygame.font.Font` pour charger notre police d'écriture typée jeu vidéo. Le menu compare dynamiquement le prix requis avec l'or actuel du joueur (`self.player.money`). Si le joueur est trop pauvre, le texte du prix passe automatiquement en rouge via un code couleur RVB (180, 40, 30) et le bouton "améliorer" est désactivé (on le dessine en gris avec une couleur "Fumée").

Création du bâtiment de défense: la tourelle

La tourelle a été une des parties les plus amusantes à créer mais aussi une des plus longues et complexes. Contrairement aux autres bâtiments comme les fermes ou les mines qui font juste monter des compteurs de ressources en arrière-plan, la tourelle devait interagir en temps réel avec des entités qui bougent sur la carte.

Sur le plan de la structure en Python, la tourelle est une instance de notre classe Bâtiment. Elle récupère donc toutes les variables de base, comme ses coordonnées de case (x, y) sur la grille, ses dimensions de hitbox et le suivi de son état de construction. C'est dans la méthode `update_attaque()` qu'elle diffère des autres bâtiment. Cette fonction est appelée à chaque frame dans la boucle principale du jeu et reçoit en paramètre la liste des ennemis actifs envoyée par notre gestionnaire de vagues (`raid_manager`).

La première étape pour la tourelle, c'est de trouver sur qui tirer. Pour ça, le code Python fait une boucle sur tous les monstres présents sur la map. Il calcule d'abord le centre exact de la tourelle en pixels, puis utilise la fonction `math.hypot(dx, dy)` du module mathématique de Python pour calculer la distance ligne droite (l'hypoténuse) entre la tourelle et chaque monstre. Le script compare ces distances pour verrouiller la cible la plus proche, à condition qu'elle soit inférieure au rayon de portée configuré dans notre dictionnaire DATA (par exemple 250 pixels).

Une fois que la tourelle a trouvé sa cible, on passe à la partie visuelle et orientation, et c'est là qu'on a pas mal galéré. Il fallait que la tourelle tourne ses canons vers le monstre pour que ce soit réaliste. On a utilisé la fonction `math.atan2(dy, dx)` pour obtenir l'angle exact en radians entre la tourelle et le monstre, qu'on convertit ensuite en degrés. Comme on n'avait pas des animations à 360 degrés pour chaque pixel, on a découpé l'espace en 8 directions cardinales (Nord, Sud, Est, Ouest, et les diagonales comme Sud-Est ou Nord-Ouest). En faisant des vérifications d'intervalles d'angles (par exemple, si l'angle est entre 22.5° et 67.5°, on est en plein Sud-Est), le code met à jour une variable de chaîne de caractères `self.direction`. Dans notre boucle de rendu, Pygame utilise cette variable comme une clé pour aller chercher le bon sprite orienté dans notre gros dictionnaire `images_batiments` et l'afficher à l'écran.

Enfin, il y avait le problème de la cadence de tir. Au début, sans protection, la tourelle attaquait à chaque frame de la boucle de jeu. Comme le jeu tourne à 60 FPS, la tourelle infligeait ses dégâts 60 fois par seconde, ce qui détruisait n'importe quel

ennemi instantanément. Pour régler ça, on a réutilisé l'horloge de Pygame. On a créé une variable d'état `self.dernier_tir` initialisée à 0. À chaque frame, la tourelle vérifie si le temps actuel renvoyé par `pygame.time.get_ticks()` moins le timestamp du dernier tir est supérieur ou égal au délai de la cadence de tir (par exemple 1000 millisecondes pour un tir par seconde). Si le chrono est bon, la tourelle déclenche sa méthode d'attaque, inflige les dégâts au monstre avec `cible_proche.take_damage(degats)`, et met à jour `self.dernier_tir` avec le temps actuel. C'est un système qui fonctionne comme un cooldown, ce qui permet d'avoir des combats équilibrés et fluides sans jamais bloquer les performances du reste du jeu.

Le skill tree

L'arbre de compétences a été pensé comme l'un des piliers stratégiques d'Overdrive. Dans un jeu de gestion/survie, si le joueur peut tout construire et tout améliorer dès les premières minutes, il n'y a plus aucun sentiment de progression et la rejouabilité devient nulle. L'objectif principal de ce système est de rythmer la partie et de forcer le joueur à faire des choix : va-t-il foncer sur les technologies militaires pour sécuriser ses arrières, ou va-t-il plutôt optimiser son économie en premier au risque de se faire déborder par les raids ?

Sur le plan du Game Design, notre arbre fonctionne avec un système de dépendances classiques. Une compétence avancée (comme le niveau d'amélioration maximal d'un bâtiment) est verrouillée tant que la compétence de base précédente n'a pas été achetée.

Pour acheter un nœud dans l'arbre, le joueur doit dépenser des points ou des ressources spécifiques accumulées en jeu, dans notre cas c'est de la vapeur. L'interface est claire : le joueur doit comprendre en un coup d'œil ce qui est débloqué (généralement affiché en surbrillance ou en vert), ce qui est achetable (accessible mais pas encore payé), et ce qui est complètement verrouillé (les nœuds sombres cachés derrière un "brouillard de technologie"). On a aussi veillé à l'ergonomie : l'ouverture de l'arbre met le jeu en pause en mode solo pour laisser au joueur le temps de réfléchir à sa stratégie sans stresser à cause des monstres qui rôdent.

2. Architecture Logique et Gestion des États en Python

Côté développement, le plus grand défi avec l'arbre de compétences, c'est de faire communiquer l'état des recherches avec les autres modules du jeu (comme le placement des structures ou le menu d'amélioration) de manière totalement fluide et sans créer de bugs de duplication ou de failles dans l'économie.

La gestion par l'objet Player et les structures d'état

Pour que le système soit le plus léger possible, on a choisi de stocker les compétences débloquées directement dans l'instance du joueur (`player.unlocked_skills`). Plutôt que de créer une usine à gaz, cette variable est une simple liste (ou un set Python) qui contient des chaînes de caractères uniques faisant office d'identifiants (par exemple : `"tourelle_unlock"`, `"farm_level_2"`, etc.).

Cette centralisation dans l'objet joueur simplifie énormément les vérifications partout ailleurs dans le projet. Par exemple, quand on essaie de poser une tourelle sur la map, le fichier `jeu.py` exécute cette simple condition :

Le module externe de plafonnement (`get_max_level`)

Pour l'évolution des bâtiments, on a créé un fichier de logique pure nommé `skill_levels.py`. À l'intérieur, la fonction `get_max_level(type_batiment)` examine la liste des compétences du joueur et renvoie dynamiquement un entier (1, 2 ou 3).

C'est ce script qui sert à empêcher de toujours améliorer. Quand le joueur ouvre le `MenuAmelioration` d'une mine, le menu appelle cette fonction. Si la mine est niveau 1 et que `get_max_level("mine")` renvoie 1 (parce que la compétence de niveau supérieur n'a pas été achetée), le bouton d'upgrade se désactive automatiquement et affiche un message clair : "Upgrade non débloqué". Cette architecture évite d'avoir à recalculer tout l'arbre à chaque action ; on interroge juste l'état du joueur à l'instant T.

3. Rendu Graphique Procédural et Machine à États sous Pygame

On a décidé de produire l'arbre avec le moteur graphique de python pour le garder intuitif et simple à comprendre et utiliser.

Cartographie et lignes de connexion

L'interface de l'arbre est gérée par le module `screens.skill_tree`. Pour lier les compétences entre elles, le code ne se contente pas d'afficher des boutons n'importe où : il calcule des positions relatives sur l'écran en fonction de la résolution actuelle. Le script utilise ensuite `pygame.draw.line` pour tracer des lignes ou des tuyaux de cuivre entre les nœuds dépendants.

La couleur de ces lignes s'adapte en temps réel : si la compétence suivante est verrouillée, le fil est dessiné dans un ton gris sombre charcoal ; si le lien est actif et que la technologie est disponible, la ligne s'illumine dans une teinte laiton ou dorée, ce qui guide visuellement le joueur dans son plan de recherche.

Un des gros pièges quand on ouvre un grand menu comme l'arbre de compétences via une touche (on a configuré la touche A sur le clavier), c'est la superposition des clics de souris. Si l'arbre s'affiche par-dessus la carte, un clic pour acheter une compétence ne doit surtout pas sélectionner une mine en arrière-plan ou faire attaquer le personnage.

Pour gérer ça proprement, on utilise le retour d'état de la fonction `afficher_skill_tree()`. Tant que l'utilisateur navigue dans l'arbre, la boucle principale du jeu lui cède le contrôle exclusif de la gestion des événements (`pygame.event.get()`). La détection des boutons se fait via la méthode `.collidepoint(sx, sy)` sur les zones rectangulaires de chaque icône technologique.

Dès qu'une compétence est achetée, le script met à jour la liste, joue un effet sonore gratifiant, et redessine l'interface mise à jour. Lorsque le joueur appuie sur Echap ou clique sur le bouton de fermeture, la fonction renvoie la liste actualisée des compétences à `jeu.py`, ferme la fenêtre, et la boucle de gameplay reprend exactement là où elle s'était arrêtée. Ce cloisonnement des états garantit qu'aucun bug d'input ne vient gâcher l'expérience de jeu.

Mécanique du sol avec le Tile system

Au début du projet, on s'est posé la question de la gestion de la map. Faire une carte gigantesque générée au tout début de la partie aurait surchargé la mémoire pour rien, surtout avec Pygame qui doit gérer le rendu de chaque élément à chaque frame. On a donc opté pour un système beaucoup plus malin : une carte évolutive où le joueur doit fabriquer et poser lui-même ses dalles de sol (les Tile) pour agrandir son territoire et pouvoir y installer ses machines.

Sur le plan fonctionnel, le sol est traité exactement comme un bâtiment classique, mais avec des propriétés minimales. D'après notre dictionnaire de configuration, le type `Batiment.TYPE_TILE` possède une taille de hitbox de `$1 \times 1$` case et ne coûte qu'un seul or à produire. C'est la brique de base du jeu : le joueur ne peut poser des générateurs, des fermes ou des tourelles que si le sol a préalablement été pavé par ces dalles de cuivre ou de laiton steampunk.

2. Intégration dans le rendu du monde (render.py)

Dans le fichier de rendu que nous avons développé (render.py), la fonction `dessiner_monde()` prend en charge l'affichage de ces dalles de manière complètement transparente à travers notre boucle sur la liste générale `batiments`.

Le mécanisme du système de cache d'images

Puisque le joueur est amené à dupliquer et aligner des centaines de dalles pour étendre sa base, réclamer à Pygame de redimensionner l'image de la dalle à chaque frame pour chaque case détruirait instantanément les performances du processeur. Pour contrer cela, notre fonction fait appel à la méthode `_get_scaled_batiment_image()` qui utilise un dictionnaire de cache local nommé `scaled_cache`.

Lorsqu'une dalle `Tile` est détectée, le script vérifie si ses dimensions pixels (`footprint_w_px` et `footprint_h_px`) associées au zoom actuel existent déjà dans les clés du cache. Si c'est le cas, l'image lissée est instantanément renvoyée et dessinée via un simple `surface_monde.blit(image, (x, y))`. Cela nous permet d'étendre la map à l'infini sans subir de chute de framerate (le jeu reste bloqué à 60 FPS).

Mode aperçu et prévisualisation du chaînage

Pour que l'extension du sol soit agréable à utiliser, la fin de la fonction `dessiner_monde()` gère l'affichage d'un "fantôme" visuel lorsque la dalle est sélectionnée dans le HUD. Le script convertit les coordonnées de la souris en cases de la grille via la fonction `souris_vers_case()`.

Grâce à la méthode intégrée `image.copy()`, on duplique temporairement le sprite de la dalle et on applique un filtre de couleur de collision. Si le joueur essaie de poser une dalle par-dessus un coffre ou un bâtiment existant, la fonction applique un masque transparent rouge (255, 0, 0, 120) avec le flag de fusion `pygame.BLEND_RGBA_MULT`. Si la case est libre mais que le joueur vise une zone trop lointaine par rapport à la position de son personnage, le code utilise la fonction `joueur_a_portee()` pour teinter le fantôme en orange (255, 140, 0, 120). Cela donne un excellent retour visuel au joueur qui sait exactement où et comment il peut étendre sa base en temps réel.

2.4 Matthieu Bourgeois

Mes tâches principales :

- **Extension du contenu : enrichissement des bâtiments et des mécaniques**
- **Adaptation et correction de problèmes techniques**

Extension du contenu : enrichissement des bâtiments et des mécaniques

Ajout des centrales de boost de production

L'une des évolutions majeures du projet a été l'introduction de bâtiments spécialisés dans l'augmentation de la production des ressources.

Ces structures, appelées centrales de boost, permettent d'améliorer les performances globales de l'économie du joueur. Leur rôle n'est pas de produire directement des ressources, mais d'augmenter l'efficacité des autres bâtiments déjà présents sur la carte.

Ce choix de design a été motivé par la volonté de donner une dimension stratégique plus profonde au jeu. Le joueur ne se contente plus de placer des bâtiments de production de manière isolée : il doit désormais réfléchir à leur optimisation spatiale et à leur synergie avec les centrales de boost.

Ainsi, ces bâtiments introduisent une logique d'optimisation progressive, où le placement et la gestion de l'espace deviennent des éléments centraux du gameplay.

Ajout des bâtiments de stockage des ressources

Dans une logique d'approfondissement du système économique, nous avons également intégré des bâtiments de stockage des ressources.

Ces structures ont pour rôle d'augmenter la capacité maximale de stockage du joueur, permettant ainsi de conserver une quantité plus importante de ressources avant d'atteindre la limite.

Avant leur ajout, le système de ressources reposait sur une capacité fixe, ce qui pouvait rapidement limiter la progression et freiner le développement du joueur. L'introduction de ces bâtiments a donc permis d'apporter une véritable dynamique de progression.

Le joueur est désormais encouragé à construire et améliorer son infrastructure non seulement pour produire davantage, mais aussi pour pouvoir stocker et gérer efficacement des quantités plus importantes de ressources sur le long terme.

Mise en place d'une mécanique de ressources maximales évolutives

En complément des bâtiments de stockage, une mécanique de ressources maximales a été introduite afin de structurer plus clairement la gestion économique du jeu.

Chaque type de ressource dispose désormais d'une limite maximale, représentant la capacité totale que le joueur peut posséder à un instant donné.

Cette limite n'est cependant pas fixe : elle peut être augmentée progressivement grâce aux bâtiments de stockage.

Cette approche apporte plusieurs avantages :

- Elle introduit une contrainte stratégique de gestion des ressources.
- Elle empêche l'accumulation illimitée sans planification.

- Elle donne une utilité directe et constante aux bâtiments de stockage, qui ne sont plus simplement optionnels mais essentiels à la progression.

Adaptation et correction de problèmes techniques

Correction du système de déplacement (pathfinding)

Un problème important a été identifié concernant le système de déplacement du joueur sur la carte.

Dans certaines situations, il devenait impossible de cliquer sur certaines zones, notamment en partie basse de l'écran, ce qui empêchait le personnage de s'y déplacer correctement. Ce dysfonctionnement affectait directement la fluidité de l'expérience de jeu.

Une analyse du comportement du système a permis de corriger ce problème, en réajustant la manière dont les coordonnées étaient interprétées lors des déplacements. Après correction, le joueur peut désormais se déplacer librement sur l'ensemble de la carte sans restriction involontaire.

Cette amélioration a grandement contribué à rendre les déplacements plus fiables et plus intuitifs.

Correction du système de sauvegarde et des problèmes de mise à l'échelle

Un autre problème majeur concerne le système de sauvegarde du jeu.

Lors du chargement d'une partie enregistrée, certains bâtiments n'étaient pas correctement restaurés visuellement. Plus précisément, leur taille (ou mise à l'échelle) pouvait être incorrecte, donnant un rendu incohérent par rapport à leur état initial.

Ce problème provenait d'une mauvaise restitution de certains paramètres lors du chargement des données de sauvegarde.

Une correction a été apportée afin de garantir que chaque bâtiment retrouve exactement son état d'origine lors de la restauration d'une partie. Cela inclut notamment :

- leur position,
- leur taille,
- et leur état fonctionnel.

2.5 Anatole Brunelle

Changement de grille

- **Comment ça fonctionnait**

Dans la première version du jeu, la carte reposait sur une grille dont chaque case mesurait exactement la taille d'un bâtiment. Le principe était simple : une case = un emplacement = un bâtiment. Pour placer une mine, on cliquait sur une case vide et elle la remplissait intégralement.

Cette approche avait l'avantage d'être facile à implémenter : la position d'un bâtiment s'exprimait par un simple index (ligne, colonne) dans un tableau 2D, et la détection de collision se résumait à vérifier si la case était occupée ou non.

- **Les problèmes de cette approche**

Plusieurs limitations importantes sont apparues rapidement :

- Les cases étaient visuellement très grandes à l'écran, ce qui donnait une sensation de carte grossière et peu détaillée, peu cohérente avec l'esthétique steampunk souhaitée.
- La taille fixe imposait que tous les bâtiments aient exactement la même emprise au sol, ce qui rendait impossible de donner à une tourelle une empreinte plus petite qu'une mine, ou d'imaginer de futurs bâtiments 2×3 ou 1×1 .

- Le nombre de cases disponibles sur la carte était directement limité par la résolution de l'écran : avec des cases de 128 px, une fenêtre 1280×720 n'offrait que 10×5 emplacements, ce qui rendait le village minuscule.
- La grille était perçue visuellement comme une contrainte rigide et lourde, rappelant plus un tableur qu'un jeu de construction.

● Principe de la nouvelle grille

La refonte remplace les grandes cases-bâtiment par une grille de petites cases de 40 px chacune. Ces cases sont purement logiques : elles forment un espace de coordonnées continu dans lequel tout se positionne (joueur, bâtiments, monstres, NPC). La carte n'a plus de bords définis — elle s'étend théoriquement à l'infini dans toutes les directions, et seule la caméra décide de ce qui est visible.

● Un bâtiment occupe plusieurs cases

Chaque bâtiment a maintenant deux attributs largeur et hauteur, exprimés en nombre de petites cases. Ces valeurs sont indépendantes pour chaque type de bâtiment et peuvent être modifiées sans toucher au reste du code.

Par défaut, tous les bâtiments principaux (maison, mine, ferme, générateur) occupent 5×5 cases, ce qui représente 200×200 px à l'écran au zoom 1:1. Les tourelles, plus petites visuellement, occupent 4×4 cases (160×160 px). Cette différence de taille est immédiatement perceptible dans le jeu et renforce la lisibilité : une tourelle prend moins de place dans le village qu'un bâtiment de production.

● Placement centré sur la case cliquée

Quand le joueur clique pour poser un bâtiment, la case sous la souris est calculée, puis le bâtiment est centré autour de cette case. Cela donne une pose intuitive : le bâtiment « s'accroche » sous le curseur, centré, ce qui ressemble au comportement de jeux comme Clash of Clans ou Clash Royale.

● Détection de collision sur la grille fine

La collision entre bâtiments est vérifiée case par case. Deux bâtiments se chevauchent si leurs rectangles de cases se superposent. Cette vérification est faite avant tout placement :

Cette approche est bien plus précise que l'ancienne vérification « la case est-elle occupée ? » car elle fonctionne quel que soit le nombre de cases occupées par chaque bâtiment. Elle permet également de détecter les collisions partielles (un bâtiment 5×5 qui mordrait le coin d'un autre).

● Grille infinie sans limite de carte

Le rendu de la grille est entièrement basé sur la position de la caméra. À chaque frame, on calcule quelle portion du monde est visible, et on dessine uniquement les lignes qui tombent dans cette zone. La grille s'étend aussi loin que le joueur scrolle, sans jamais atteindre un bord.

Le zoom est géré au niveau de l'écran (post-render) plutôt que de la grille elle-même, ce qui évite les artefacts de sub-pixel. Quand le zoom est très petit (< 2 px entre deux traits), la grille n'est tout simplement pas dessinée pour éviter le bruit visuel.

● L'ancien système, tuiles répétées

Dans la version précédente du jeu, le fond de carte était rendu en répétant une petite image de texture (grass.png, 225×225 px) en mosaïque sur toute la surface visible. À chaque frame, le moteur calculait combien de tuiles tenaient dans la fenêtre et appelait `surface.blit()` autant de fois que nécessaire pour couvrir la vue.

Sur une fenêtre 1280×720, avec des tuiles de 225 px, cela représentait environ $6 \times 4 = 24$ appels blit par frame uniquement pour le fond, auxquels s'ajoutaient les décalages de caméra à recalculer à chaque fois. Plus la fenêtre était grande ou le zoom dézoomé, plus le nombre de tuiles augmentait.

● La nouvelle approche, une grande image unique

La refonte remplace le système de tuiles par une seule grande image de fond (background.png, 728×437 px) qui est scalée pour couvrir toute la surface de rendu en un seul blit. Quel que soit le zoom ou la taille de la fenêtre, le coût du rendu du fond est constant : un seul `pygame.transform.scale()` suivi d'un seul blit.

Le fond est dessiné sur `surface_monde` (la surface intermédiaire de rendu mondial) avant tous les autres éléments, puis la `surface_monde` entière est ensuite scalée vers l'écran. Cela signifie que le fond bénéficie automatiquement du pipeline d'optimisation déjà en place.

La grille de cases (les lignes fines dorées) est dessinée par-dessus en post-processing, directement en coordonnées écran après le scaling. Cela évite les artefacts classiques où les lignes de grille deviennent floues ou décalées d'un pixel selon le niveau de zoom.

● La surface intermédiaire `surface_monde`

En parallèle de la refonte du fond, un pipeline de rendu à deux étapes a été introduit pour optimiser l'ensemble du rendu du monde, pas seulement le fond :

- Étape 1 — Rendu monde : tout le monde (fond, bâtiments, joueur, NPC, monstres) est dessiné sur `surface_monde`, une surface Pygame à la résolution du monde visible ($1280/\text{zoom} \times 720/\text{zoom}$ px).
- Étape 2 — Scaling écran : `surface_monde` est scalée en un seul appel `pygame.transform.scale()` vers la résolution de l'écran réel.

L'avantage est double : d'une part, tous les sprites et éléments sont dessinés à la résolution naturelle du monde (pas besoin de scaler chaque sprite individuellement), et d'autre part, le zoom ne coûte qu'un seul `transform.scale` au lieu de N transforms pour N entités.

La `surface_monde` est recrée uniquement quand sa taille change (changement de zoom ou redimensionnement de fenêtre), et non à chaque frame. Cela évite des allocations mémoire répétées qui auraient un impact sur les performances.

Attribution manuelle des villageois

● Présentation

Le jeu dispose maintenant d'un menu dédié à la gestion des villageois, accessible à tout moment en appuyant sur TAB. Ce menu permet au joueur de décider précisément

quel villageois travaille dans quel bâtiment, au lieu de laisser la répartition se faire automatiquement.

● Interface

Le menu s'affiche en superposition sur le jeu, avec un fond semi-transparent. Il est organisé en deux colonnes scrollables :

- À gauche : la liste de tous les villageois présents dans le village, avec leur état actuel (en errance, en déplacement, au travail) et leur lieu de travail courant.
- À droite : la liste de tous les bâtiments de production disponibles (mines, fermes, générateurs), avec le nombre de villageois déjà assignés.

Le thème graphique adopte une palette bois et cuivre cohérente avec l'esthétique steampunk du jeu : fonds sombres, cartes en beige chaud, textes dorés.

● Fonctionnement

L'assignation se fait en deux clics : on clique d'abord sur un villageois dans la colonne gauche pour le sélectionner (il se surligne), puis on clique sur le bâtiment cible dans la colonne droite. L'assignation est immédiate et persiste jusqu'à destruction du bâtiment ou réinitialisation manuelle.

La couleur de fond de chaque carte de villageois donne une information d'un coup d'œil :

- Vert sombre : villageois assigné manuellement à un bâtiment spécifique.
- Rouge sombre : le bâtiment cible est déjà à pleine capacité.
- Beige sélectionné : villageois en attente d'une cible.
- Beige neutre : villageois en répartition automatique (round-robin).

● Boutons d'action

Trois boutons globaux complètent l'interface :

- Auto-assigner : redistribue tous les villageois automatiquement sur les bâtiments disponibles, en annulant les assignations manuelles.
- Tout effacer : libère tous les villageois de leur poste. Ils repassent en errance jusqu'à la prochaine synchronisation.

- Fermer (ou TAB / Échap) : ferme le menu sans modifier les assignations.

● Robustesse

Si un bâtiment assigné à un villageois est détruit pendant la partie, l'assignation est annulée proprement et le villageois revient en répartition automatique. De même, si la maison d'un villageois est détruite, son entrée est supprimée de la table d'assignation pour éviter toute fuite mémoire.

Terminal développeur — accès restreint

● Le terminal

Le jeu intègre un terminal de développement, accessible via la touche ² (tilde sur clavier AZERTY). C'est une console en overlay qui permet d'exécuter des commandes directement pendant la partie : déclencher un raid, donner des ressources, afficher des informations de debug.

Parmi les commandes disponibles :

- give <ressource> <quantité> — Ajoute de l'or, de la nourriture ou de la vapeur.
- godlike — Donne instantanément 10 000 de chaque ressource.
- trigger_raid — Déclenche immédiatement une vague de monstres.
- help — Affiche toutes les commandes disponibles.
- hello world — Easter egg : déclenche une animation Matrix Rain sur tout l'écran pendant 4 secondes.

● Restriction au mode développeur

On a ajouté un mode développeur pour tester notre jeu, celui ci nous donne accès au terminal

Le mode développeur modifie également d'autres comportements pour faciliter les tests :

- La sauvegarde existante est ignorée : on repart d'une partie vierge.
- Le tutoriel ne s'affiche pas.
- Le joueur démarre avec 5 000 de chaque ressource.

- La connexion réseau multijoueur est ignorée.

Cette séparation claire entre mode normal et mode développeur empêche toute utilisation involontaire (ou intentionnellement abusive) des commandes de triche en partie standard.

Ergonomie du placement de bâtiments

● Raccourcis clavier 1 à 5

Il est maintenant possible de sélectionner un type de bâtiment directement au clavier, sans passer par la barre d'interface en bas de l'écran :

Touche	Bâtiment sélectionné
1	Maison (bâtiment résidentiel, génère des villageois)
2	Générateur (produit de la vapeur)
3	Mine (produit de l'or)
4	Ferme (produit de la nourriture)
5	Tourelle (défense automatique contre les monstres)
6	Chemin

● Placement continu

Une fois un bâtiment sélectionné, le joueur peut en poser plusieurs à la suite sans avoir à re-sélectionner entre chaque pose. La sélection reste active jusqu'à ce que le joueur la désactive explicitement (clic droit ou clic sur la croix dans la barre). Cela rend la construction de longues lignes de tourelles ou de quartiers résidentiels beaucoup plus fluide.

Corrections de stabilité

● Apparition des villageois

Les villageois apparaissent parfois à des positions incorrectes ou traversent des cases invalides au moment de leur création. Ils démarrent maintenant systématiquement au centre de leur maison, dans un état d'errance calme, avant de commencer à se déplacer vers leur lieu de travail. Cela évite les téléportations visuelles et les villageois hors carte.

● Messages flottants

Avant, si on ne pouvait pas placer un bâtiment, rien ne nous indiquait la cause, alors j'ai ajouté des messages flottants pour indiquer la raison. Seulement, après l'ajout du placement continu, les messages étaient spammés, donc il a fallu ajouter un délai entre chaque message.

● Déplacement des villageois sur la grille

Les villageois peuvent maintenant uniquement se déplacer sur des cases de sol valides. Auparavant, les cibles de déplacement aléatoire (errance) pouvaient tomber en dehors de la grille ou sur des cases de décor, ce qui provoque des comportements étranges. Les positions cibles sont désormais systématiquement filtrées pour rester dans les limites de la zone jouable.

● Correction du chemin vers le lieu de travail

Quand un villageois se trouvait déjà sur la case de son bâtiment de destination, le calcul de chemin retournait une liste vide, ce qui provoquait une micro-téléportation. Le système détecte maintenant ce cas et passe directement le villageois en état "au travail" sans calculer de déplacement inutile.

Conclusion

Au terme de ce semestre de développement, le projet Overdrive représente une réalisation dont l'équipe peut être fière. Partis d'un cahier des charges exigeant — un jeu multijoueur en réseau, avec gestion de ressources, bâtiments améliorables, intelligence artificielle et ambiance complète —, nous avons livré un produit fonctionnel, jouable et techniquement consistant. Ce résultat est le fruit d'un travail collectif rigoureux, où chaque membre a su prendre en charge son périmètre tout en restant disponible pour les autres.

Sur le plan technique, ce projet nous a confrontés à des problématiques rarement abordées dans les exercices académiques classiques. L'implémentation d'un module réseau TCP/UDP avec synchronisation en temps réel, la conception d'un pipeline de rendu optimisé avec surface intermédiaire, la gestion d'un système de pathfinding pour des PNJ autonomes, ou encore l'architecture d'un arbre de compétences interactif : chacun de ces chantiers a nécessité une véritable réflexion de conception, bien au-delà de la simple mise en œuvre. Nous avons appris à identifier les bonnes abstractions, à anticiper les interactions entre les modules et à documenter nos choix pour faciliter la collaboration au sein de l'équipe.

Ce projet nous a également appris à composer avec les limitations d'un environnement technique contraint. Python et Pygame ne sont pas les outils naturellement prédestinés à ce type de jeu, et plusieurs fonctionnalités ont demandé des contournements ingénieux : gestion de threads pour le réseau sans bloquer la boucle de jeu, système de cache d'images pour maintenir un framerate stable malgré la densité des éléments affichés, ou encore architecture orientée objet rigoureuse pour rendre les modules indépendants et testables séparément. Ces contraintes, loin d'être des obstacles, nous ont poussés à développer un sens aigu de l'optimisation et de la rigueur architecturale. Au-delà des aspects purement techniques, ce projet a été une expérience formatrice sur le plan humain et organisationnel. Travailler en équipe sur un logiciel de cette envergure exige une coordination permanente : il a fallu définir des interfaces claires entre les modules, gérer les conflits de version, s'assurer que l'ajout d'une fonctionnalité par l'un ne cassait pas le travail d'un autre, et maintenir une vision commune du produit tout au long du développement. Ces compétences de collaboration et de communication sont aussi précieuses que les compétences techniques, et nous avons eu l'occasion de les exercer concrètement au fil des semaines.

Si nous devons identifier les axes d'amélioration les plus importants pour une éventuelle suite du projet, plusieurs pistes se dégagent naturellement des bilans individuels présentés dans ce rapport. Le module réseau gagnerait à être complété par un système d'interpolation côté client pour lisser les déplacements en cas de latence, et par un protocole UDP dédié aux mises à jour de position. Le système de PNJ pourrait être enrichi d'un comportement plus diversifié, avec des rôles distincts selon le type de bâtiment auquel le villageois est assigné. L'équilibrage du jeu, notamment la progression économique et la difficulté des vagues, mériterait également des tests plus systématiques avec des joueurs extérieurs. Ces perspectives montrent que le

projet est loin d'être figé et pourrait facilement évoluer vers un produit encore plus abouti.

En définitive, Overdrive est un projet dont nous retirons une grande satisfaction. Il nous a permis de mettre en pratique, sur un cas concret et exigeant, les connaissances acquises tout au long de notre formation, tout en découvrant de nouvelles problématiques liées au développement de jeux vidéo. Chaque membre de l'équipe repart avec des compétences concrètes, une meilleure compréhension de l'architecture logicielle, et la conviction que des réalisations ambitieuses sont possibles lorsque le travail est bien organisé et l'engagement de chacun au rendez-vous.